

Rapport de veille technologique backend

NestJS

NestJS est un framework pour développer des applications serveur avec Node.js et TypeScript. Son intérêt est de donner une place aux différentes parties du code : les contrôleurs reçoivent les requêtes, les services traitent les règles métier et les modules regroupent les fonctionnalités. Les dépendances sont fournies par le conteneur Nest, plutôt que créées directement dans chaque classe. Cette organisation permet notamment de remplacer un service par un mock pendant les tests. [\[1\]](#)

Pour une API comme le Kanban, ce cadre aide à garder la même logique d'organisation entre les utilisateurs, les listes et les cartes. L'intégration Swagger permet aussi de générer une documentation OpenAPI et de tester les routes depuis le navigateur. Elle demande toutefois de renseigner correctement les schémas et les réponses : la documentation ne devient pas complète toute seule. [\[2\]](#)

Le démarrage demande un effort pour comprendre les décorateurs, l'injection et les imports de modules. Il faut aussi choisir les bibliothèques de persistance et d'authentification. Enfin, Node.js impose de faire attention aux calculs longs sur le thread principal, qui peuvent retarder les autres requêtes. [\[3\]](#) Je retiens donc surtout NestJS pour une API TypeScript structurée, avec une équipe prête à apprendre ses conventions.

Sources

- [1 NestJS : providers et injection de dépendances.](#)
- [2 NestJS : intégration OpenAPI et Swagger.](#)
- [3 Node.js : éviter de bloquer la boucle événementielle.](#)

Symfony

Symfony est un framework PHP construit autour de composants réutilisables. Son conteneur de services gère les dépendances de l'application ; l'autowiring permet notamment de les injecter à partir des types déclarés. [\[1\]](#) On retrouve ainsi une séparation entre contrôleurs et services comparable à celle de NestJS, même si le langage et les outils changent.

Pour le Kanban, les voters seraient une façon de centraliser les décisions d'accès : un utilisateur peut-il modifier cette liste ou cette carte ? Ils permettent d'évaluer une permission sur un objet donné. [\[2\]](#) Cette approche m'intéresse parce qu'elle distingue la connexion de l'utilisateur de son droit à agir sur une ressource précise.

Un autre argument est la visibilité sur la maintenance. Symfony publie un calendrier de versions et prévoit, pour ses versions LTS, trois ans de corrections et quatre ans de

correctifs de sécurité. [3] C'est utile pour anticiper les mises à jour d'un produit maintenu plusieurs années.

En revanche, une équipe venant de JavaScript doit apprendre PHP, Composer et les conventions du framework. Ce coût dépend de son expérience : Symfony ne serait pas forcément plus compliqué pour des développeurs PHP. Je le considérerais donc sérieusement dans ce contexte, sans prétendre qu'il serait moins performant ou moins adapté aux API.

Sources

- [1 Symfony : conteneur de services et autowiring.](#)
- [2 Symfony : contrôle des permissions avec les voters.](#)
- [3 Symfony : calendrier des versions et maintenance LTS.](#)

Spring Boot

Spring Boot facilite la création d'applications dans l'écosystème Java et Spring. L'auto-configuration prépare certains composants selon les dépendances présentes, tout en permettant de remplacer les réglages proposés. [1] Cela évite une partie de la configuration de départ, mais il faut comprendre ce qui est activé pour diagnostiquer un comportement inattendu.

Comme dans NestJS et Symfony, les contrôleurs peuvent déléguer le traitement aux services. Le conteneur Spring fournit les objets nécessaires, appelés beans, par injection de dépendances. [2] Pour une équipe, cette structure donne des repères communs entre les fonctionnalités.

Spring Boot propose aussi des outils pour suivre l'application après son déploiement. Actuator fournit notamment des endpoints de santé et de métriques, dont l'exposition et la protection doivent être configurées. [3] Cet aspect compte pour un produit qui doit être exploité dans la durée.

La difficulté, pour quelqu'un qui découvre cet environnement, est d'apprendre ensemble Java, les outils de construction et les mécanismes Spring. Je ne conclurais pas pour autant que Spring Boot est trop lourd pour une petite API. Sa consommation doit être mesurée dans un contexte précis. Les images natives peuvent réduire le démarrage et la mémoire, mais ajoutent des contraintes de compilation. [4] Une équipe déjà habituée à Java aurait de bonnes raisons de le retenir.

Sources

- [1 Spring Boot : fonctionnement de l'auto-configuration.](#)
- [2 Spring Framework : conteneur IoC et beans.](#)
- [3 Spring Boot : endpoints Actuator.](#)
- [4 Spring Boot : possibilités et contraintes des images natives GraalVM.](#)

Pourquoi j'ai choisi NestJS

J'ai choisi NestJS pour réaliser cette API Kanban en TypeScript avec une structure que je pouvais reprendre d'une fonctionnalité à l'autre. L'application est organisée en modules d'authentification, d'utilisateurs, de listes et de cartes. Les contrôleurs exposent les routes et les services portent les règles métier. Cette séparation m'a donné un cadre pour avancer, puis écrire les tests.

L'apprentissage n'a pas été immédiat. J'ai notamment rencontré des difficultés avec les imports, la génération du client Prisma et la distinction entre PostgreSQL local et la base Docker. Ces problèmes m'ont poussé à mieux documenter le lancement du projet.

Dans l'implémentation, le guard vérifie le JWT ; les services contrôlent ensuite les rôles ou la propriété des ressources. Les DTO et le pipe de validation encadrent les entrées. [1]
Swagger permet de présenter et d'essayer les routes, ce qui est utile pour vérifier le contrat et préparer la démonstration. [2]

Mon choix reste lié à ce projet. Je n'ai pas mesuré trois implémentations équivalentes et je ne peux pas affirmer que NestJS serait toujours plus rapide. Pour une équipe déjà expérimentée en PHP ou Java, Symfony ou Spring Boot pourraient être plus pertinents. Ici, NestJS m'a permis de construire une API organisée, documentée et testable.

Sources et éléments du projet

- [1 NestJS : validation des requêtes.](#)
- [2 NestJS : documentation OpenAPI.](#)